VIETNAM NATIONAL UNIVERSITY, HANOI

HANOI SCHOOL OF BUSINESS AND MANAGEMENT

----------o0o----------

FINAL ASSIGNMENT REPORT

Programming and Application of Information Technology in business

VIỆT HÀN WMS

A Warehouse Management System for Material and Inventory Operations

Lecturer : PhD. Ngo Trung Kien

Code: HSB2006E

Class: MET5

Group: 01

Hanoi, 2026

TEAM MEMBERS

Members	Student ID	Contribution
VŨ QUANG VINH	23080093	100%
LÊ THẢO LINH	23080054	100%
NGUYỄN HỒNG VƯỢNG	23080094	100%
TRẦN HẢI ĐĂNG	22080019	100%
NGUYỄN MINH CHIẾN	23080013	100%
TRƯƠNG THUỲ LINH	23080055	100%

TABLE OF CONTENTS

1. Cover page and Project identification

This report presents the design, implementation, and evaluation of VIỆT HÀN WMS, a database-driven web application developed for the HSB2006 - Developing Business Applications final examination at the Hanoi School of Business and Management, Vietnam National University, Hanoi.

VIỆT HÀN WMS is designed to support material and inventory management for a small, single-warehouse operation. The system centralizes product, category, and supplier data, manages stock-in and stock-out transactions with automatic inventory adjustment, and provides supporting functions including role-based authentication, dashboard visualization, in-application notifications, Excel-based bulk import, inventory reporting, and email-based OTP password reset.

The system is implemented using PHP and SQLite and was verified through direct inspection of the submitted source code and database. This report documents the system architecture, database design, functional and non-functional requirements, security controls, testing approach, installation procedure, and requirement traceability. Outstanding evidence, including formal UML diagrams, executed test results, screenshots, repository history, and student identification details, should be completed before final submission.

2. Executive summary and Business problem

Warehouses that rely on manual records or spreadsheets often face difficulties in maintaining accurate and up-to-date inventory information. Common problems include inconsistent data entry, delayed visibility of stock levels, and difficulty tracking the history of stock-in and stock-out transactions. These limitations can lead to inventory discrepancies and reduce the efficiency of warehouse operations (Atieh et al., 2016).

VIỆT HÀN WMS was developed to address these problems by digitalizing and centralizing warehouse information within a single system. It provides structured management of products, categories, suppliers, and inventory transactions while automatically updating stock quantities when goods are received or issued. By replacing fragmented manual records with a centralized database and controlled transaction process, the system aims to improve inventory accuracy, traceability, and operational efficiency for small warehouse operations.

3. Project scope, Users, Assumptions, and Limitations

3.1 Scope

The scope of VIỆT HÀN WMS, as verified in the source code, covers a single-warehouse, single-location operation. In scope: category, product, and supplier master data management; stock-in and stock-out transactions with automatic quantity adjustment; inventory reporting; a statistics dashboard; user account management; role-based authorization for two roles; in-app notifications; bulk import of categories, products, and suppliers from Excel files; and a forgot-password flow using an emailed OTP. Out of scope: multi-warehouse or multi-location inventory, barcode/RFID scanning, integration with external ERP/accounting systems, and purchase-order or sales-order management beyond the receipt/issue documents already described.

3.2 Users

The database confirms exactly two roles, stored in the roles table: Admin (role_id = 1) and Nhân viên / Staff (role_id = 2). Seven user accounts exist in the demonstration database, distributed across both roles.

Table 3.1. User roles and Verified permissions

Role	Typical permissions (verified against code)
Admin	Full CRUD on categories, products, and suppliers; Excel import; user/account management; access to all reporting and dashboard views
Staff	View master data, create stock-in and stock-out transactions, and view inventory reports. restricted from administrative CRUD and import functions

3.3 Assumptions

The development and operation of VIỆT HÀN WMS are based on several assumptions regarding its execution environment and intended usage. The application is assumed to run on a single machine using the PHP built-in development server (php -S) rather than a production web server such as Apache or Nginx. The SQLite database is also assumed to be accessed by a single application instance at a time, as the current architecture was developed for local demonstration rather than concurrent multi-user production workloads. In addition, the password-reset function relies on outbound email

delivery and therefore assumes that the system has a working Internet connection and access to a valid and authorised SMTP account for sending one-time passwords (OTPs)

3.4 Limitations

Despite providing the core functionality required for warehouse and inventory management, VIỆT HÀN WMS has several technical and operational limitations. The backend is implemented using a procedural and monolithic PHP structure rather than an MVC or layered architecture, meaning that individual files such as products.php may combine request handling, database access, validation, and HTML rendering. Furthermore, Cross-Site Request Forgery (CSRF) protection is currently implemented only in account.php and has not been applied consistently across all state-changing CRUD operations. The use of SQLite is appropriate for local demonstration purposes but limits the system's suitability for high-concurrency production environments. No automated unit or integration testing suite was identified in the submitted source code, which means that testing currently relies primarily on manual verification. The codebase also contains development artifacts, including backup files, .bak and .before_* variants, and PowerShell patch scripts, which should be removed from a production release. Finally, a live or hosted deployment link was not available at the time of documentation; therefore, the application was demonstrated in a local environment.

4. Functional and Non-Functional requirements

4.1 Functional requirements

ID	Requirement	Verified in
FR-01	The system shall allow a new user to register an account, selecting either the employee or admin role using a security code.	register.php
FR-02	The system shall authenticate a user by username and password and establish a session on success.	login.php, password_verify()
FR-03	The system shall restrict administrative functions (category/product/supplier CRUD, import, and user management) to the Admin role.	config/auth.php - requireAdmin()
FR-04	The system shall allow an authorized user to create, view, update, and delete product categories, subject to a rule that a category in use by existing products cannot be deleted.	categories.php

ID	Requirement	Verified in
FR-05	The system shall allow an authorized user to create, view, update, and delete products, subject to a rule that a product referenced by existing receipt or issue records cannot be deleted.	products.php
FR-06	The system shall allow an authorized user to create, view, update, and delete suppliers, subject to a rule that a supplier referenced by existing products cannot be deleted.	suppliers.php
FR-07	The system shall allow a user to record a stock-in (receipt) transaction for one or more products from a supplier and shall increase the corresponding product quantities accordingly.	stock_in.php
FR-08	The system shall allow a user to record a stock-out (issue) transaction for one or more products and shall decrease the corresponding product quantities only if sufficient stock is available.	stock_out.php
FR-09	The system shall display a dashboard summarizing recent transactions, recent products, and a chart of receipts/issues over a selectable period.	index.php, chart_data.php
FR-10	The system shall generate an in-app notification whenever a stock-in or stock-out transaction is created.	createNotification(), notifications table
FR-11	The system shall allow bulk import of categories, products, and suppliers from Excel files, presenting a preview for user confirmation before committing the data.	import_products.php, import_categories.php, import_suppliers.php
FR-12	The system shall allow a user who has forgotten their password to reset it via a one-time password sent to their registered email address.	forgot-password.php
FR-13	The system shall record an inventory report showing current stock levels by product.	inventory_report.php
FR-14	The system shall log significant user actions to an activity log.	activity_logs table, config/logger.php

4.2 Non-Functional requirements

Table 4.2. Non-Functional requirements

ID	Category	Requirement	Verified evidence
NFR-01	Security	Passwords shall not be stored in plaintext.	password_hash()/password_verify() in users table
NFR-02	Security	Database queries built from user input shall use parameterized statements.	$pdo->prepare()/execute() used throughout
NFR-03	Data integrity	Stock-out shall never reduce a product's quantity below zero, even under concurrent requests.	conditional UPDATE ... WHERE quantity >= ? in stock_out.php
NFR-04	Reliability	Multi-table write operations (receipt/issue header + line items + quantity update) shall be atomic.	beginTransaction()/commit()/rollBack()
NFR-05	Usability	The interface shall present role-appropriate navigation and provide validation feedback for CRUD forms.	Partially verified; formal UI/UX evidence pending screenshots [MISSING EVIDENCE]
NFR-06	Portability	The system shall be runnable without a separately installed database server.	SQLite embedded database file

5. User stories, Acceptance criteria, and Project board

Table 5.1. User stories and Draft acceptance criteria

ID	User story	Acceptance criteria (draft)
US-01	As an Admin, I want to log in with my username and password so that I can access administrative functions.	Given valid credentials, the user is redirected to the dashboard; given invalid credentials, an error is shown and no session is created.

ID	User story	Acceptance criteria (draft)
US-02	As a staff member, I want to record a stock-out transaction so that product quantities are reduced accurately.	Given a requested quantity greater than available stock, the transaction is rejected and no quantity change occurs.
US-03	As an Admin, I want to import a list of products from Excel so that I do not have to enter them one by one.	Given a file with invalid or duplicate rows, the system flags them in a preview before any data is written to the database.
US-04	As a user, I want to reset my password if I forget it so that I can regain access to my account.	Given a valid registered email, an OTP is sent and must be entered correctly, within its validity window, before a new password is accepted.

6. System diagrams and Data dictionary

The diagrams in this section have been reconstructed by the author of this report from the verified control flow of the source code. They are presented here as textual/structural descriptions with placeholders for the formal UML renderings that should be produced with a diagramming tool (e.g., draw.io, Lucidchart, StarUML) and inserted before submission, since the examination requires diagrams that are consistent with the implemented application.

6.1 Use case diagram

Actors: Admin, staff and an implicit external mail server (Gmail SMTP, used only for the forgot-password flow).

Table 6.1. Use cases and Primary actors

Use case	Primary actor(s)	Verified in
Register an account.	Admin, Staff	register.php
Log in / Log out	Admin, Staff	login.php, logout.php
Reset password through OTP	Admin, Staff (+ Mail Server)	forgot-password.php
Manage categories	Admin	categories.php

Use case	Primary actor(s)	Verified in
Manage products	Admin	products.php
Manage suppliers	Admin	suppliers.php
Import Excel data	Admin	import_products.php, import_categories.php, import_suppliers.php
Record stock-in	Admin, Staff	stock_in.php
Record stock-out	Admin, Staff	stock_out.php
View inventory report	Admin, Staff	inventory_report.php
View dashboard / chart	Admin, Staff	index.php, chart_data.php
Receive notification	Admin, Staff	notifications_api.php, notifications.js

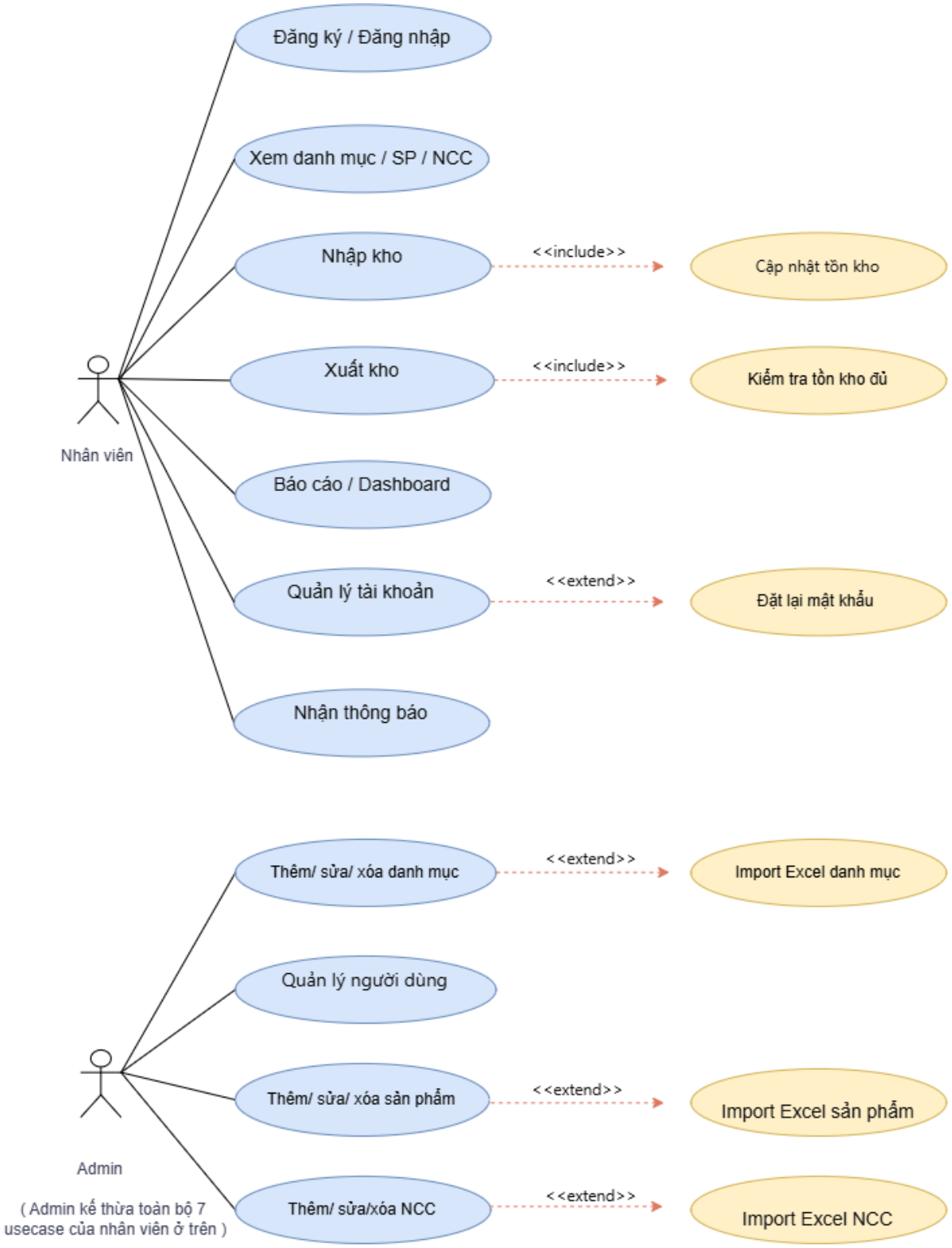
Đăng ký / Đăng nhập
Xem danh mục / SP / NCC
Nhập kho
<<include>>
Cập nhật tồn kho
Xuất kho
<<include>>
Kiểm tra tồn kho đủ
Nhân viên
Báo cáo / Dashboard
Quản lý tài khoản
<<extend>>
Đặt lại mật khẩu
Nhận thông báo
Thêm/ sửa/ xóa danh mục
<<extend>>
Import Excel danh mục
Quản lý người dùng
Thêm/ sửa/ xóa sản phẩm
<<extend>>
Import Excel sản phẩm
Admin
(Admin kế thừa toàn bộ 7 usecase của nhân viên ở trên)
Thêm/ sửa/xóa NCC
<<extend>>
Import Excel NCC

Figure 6.1. Use case diagram

6.2 Activity diagram - stock-out process

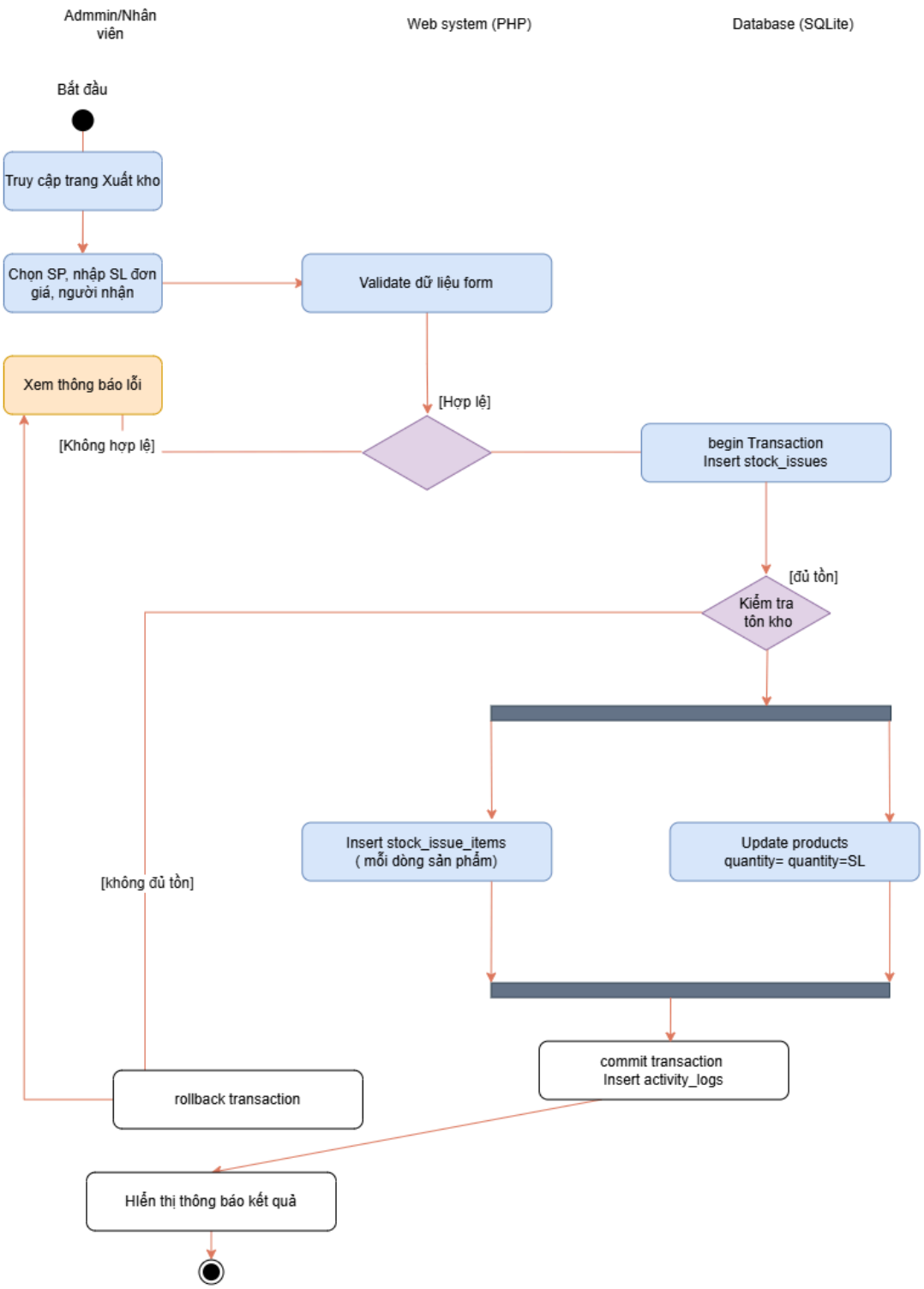

Figure 6.2. Activity diagram - stock-out process

6.3 Sequence diagram - stock-in process

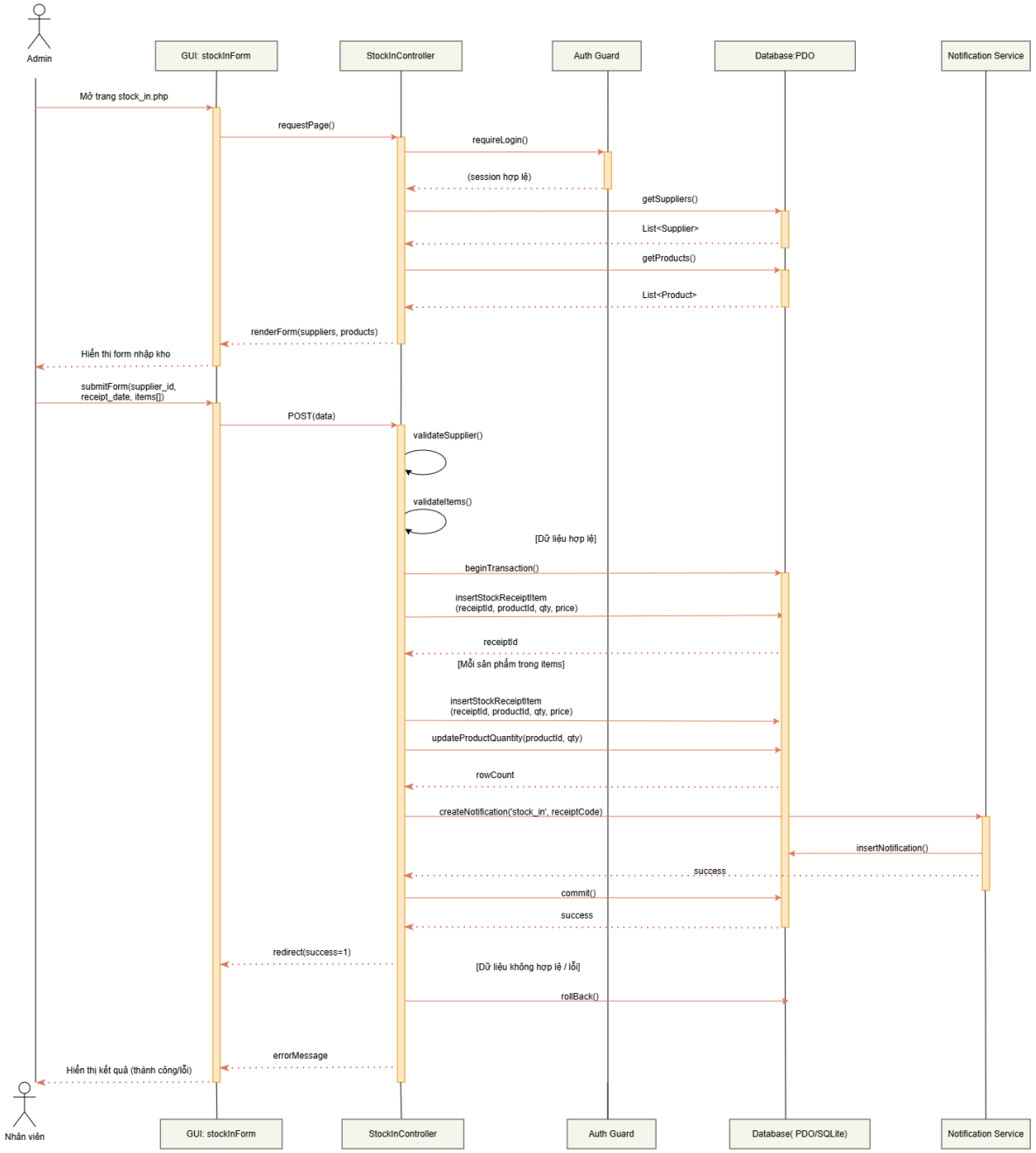

Figure 6.3. Sequence diagram - stock-in process

6.4 Data dictionary

The data dictionary below was extracted directly from the submitted SQLite database (quan_ly_kho.sqlite) using PRAGMA table_info() and PRAGMA foreign_key_list(), and therefore reflects the actual implemented schema rather than a proposed design.

Table 6.2. Data dictionary

Table	Key columns	Notes
roles	id (PK), role_name	2 rows: Admin, Nhân viên
users	id (PK), full_name, username, password, role_id (FK→roles.id), email, avatar, created_at	password stored as a password_hash() digest
categories	id (PK), category_name, description	28 rows in the demonstration database
suppliers	id (PK), supplier_name, phone, email, address	7 rows
products	id (PK), product_code, product_name, category_id (FK→categories.id), supplier_id (FK→suppliers.id), unit, import_price, sale_price, quantity, created_at	7 rows
stock_receipts	id (PK), receipt_code, supplier_id (FK), created_by, receipt_date, note, created_at	Header record for stock-in; 11 rows
stock_receipt_items	id (PK), receipt_id (FK→stock_receipts.id), product_id (FK→products.id), quantity, unit_price	Detail/line-item record; 13 rows
stock_issues	id (PK), issue_code, created_by, issue_date, recipient, note, created_at	Header record for stock-out; 7 rows
stock_issue_items	id (PK), issue_id (FK→stock_issues.id), product_id (FK→products.id), quantity, unit_price	Detail/line-item record; 7 rows
notifications	id (PK), user_id, type, title, message, is_read, created_at	63 rows in the demonstration database
activity_logs	id (PK), user_id, action, created_at	98 rows

The database follows a header-detail (master-detail) pattern for both inbound and outbound transactions: a single stock_receipts or stock_issues header record is associated with one or more corresponding *_items rows, each referencing a single product. Referential integrity is enforced at the SQLite level via PRAGMA foreign_keys = ON, and all foreign key relationships confirmed above were verified programmatically rather than assumed from documentation.

7. Application architecture, Technology stack, and Database schema

7.1 Architecture

VIỆT HÀN WMS follows a monolithic, procedural, server-side-rendering architecture. It does not use a PHP MVC framework such as Laravel or Symfony, and it does not use a JavaScript SPA framework such as React, Vue, or Angular. Each PHP file under app/ (for example products.php, stock_in.php) is responsible for handling the HTTP request, performing validation, executing database queries through PDO, and rendering the resulting HTML in the same file. This is a recognized architectural pattern for small-scale procedural web applications, though it differs from a layered or MVC design in that concerns are not separated into distinct controller, service, and view components.

Two endpoints act as lightweight JSON APIs consumed by client-side JavaScript: chart_data.php, which aggregates receipt and issue counts over a selectable period (7 days, 30 days, or the current month) for the dashboard chart, and notifications_api.php, which supports polling-based retrieval of unread notification counts and marking notifications as read. The client polls notifications_api.php on a fixed interval rather than using a persistent connection such as a WebSocket.

7.2 Technology stack

Table 7.1. Technology stack

Layer	Technology	Verified evidence
Server-side language	PHP	app/*.php
Web server (development)	PHP built-in development server	CHAY_WEB.bat: php -S localhost:8000 -t app
Database	SQLite, accessed via PDO	app/database/quan_ly_kho.sqlite, app/config/database.php

Layer	Technology	Verified evidence
Dependency management	Composer	composer.json / composer.lock in app/
Email delivery	PHPMailer 7.1.1 over SMTP	composer.lock; app/config/mail.php
Spreadsheet import	PhpSpreadsheet 5.9.0	composer.lock; import_*.php
Client-side charting	Chart.js (CDN)	index.php script tag
Client-side scripting	Vanilla JavaScript	notifications.js

7.3. Data dictionary

The VIET-HAN WMS database is implemented on SQLite and consists of 10 business tables, organized according to the relational model with Primary Key, Foreign Key, and Unique constraints that correspond precisely to the entities appearing in the Use Case, Sequence, and Activity Diagrams (e.g., stock_receipts/stock_receipt_items correspond to the Stock-in Sequence Diagram; stock_issues/stock_issue_items correspond to the Stock-out Activity Diagram).

7.3.1. Table roles - User Roles

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Role identifier
role_name	TEXT	NOT NULL, UNIQUE	Role name (Admin, Staff)

7.3.2. Table users - System Users

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	User identifier
full_name	TEXT	NOT NULL	Full name
username	TEXT	NOT NULL, UNIQUE	Login username

password	TEXT	NOT NULL	Hashed password
role_id	INTEGER	NOT NULL, FK → roles(id)	User's assigned role
email	TEXT	-	Email address (used for password-reset OTP)
avatar	TEXT	-	Path to profile avatar image
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

7.3.3. Table categories — Product Categories

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Category identifier
category_name	TEXT	NOT NULL	Category name
description	TEXT	-	Detailed description

7.3.4. Table suppliers — Suppliers

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Supplier identifier
supplier_name	TEXT	NOT NULL	Supplier name
phone	TEXT	-	Contact phone number
email	TEXT	-	Contact email
address	TEXT	-	Address

7.3.5. Table products — Products

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Product identifier
product_code	TEXT	NOT NULL, UNIQUE	Product code
product_name	TEXT	NOT NULL	Product name
category_id	INTEGER	FK → categories(id)	Product's category
supplier_id	INTEGER	FK → suppliers(id)	Default supplier
unit	TEXT	NOT NULL	Unit of measure
import_price	REAL	DEFAULT 0	Purchase/import price
sale_price	REAL	DEFAULT 0	Sale price
quantity	INTEGER	DEFAULT 0	Current stock-on-hand -automatically updated by the Stock-in/Stock-out operations (updateProductQuantity)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

7.3.6. Table stock_receipts - Stock-in Receipt (Document Header)

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Receipt identifier
receipt_code	TEXT	NOT NULL, UNIQUE	Stock-receipt code
supplier_id	INTEGER	FK → suppliers(id)	Supplier for this batch
created_by	INTEGER	FK → users(id)	User who created the receipt (audit)
receipt_date	DATE	NOT NULL	Date of stock receipt
note	TEXT	-	Notes
created_at	DATETIME	-	Record creation timestamp

7.3.7. Table stock_receipt_items - Stock-in Line Items

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Line-item identifier
receipt_id	INTEGER	NOT NULL, FK → stock_receipts(id)	Parent receipt
product_id	INTEGER	NOT NULL, FK → products(id)	Product received
quantity	INTEGER	NOT NULL	Quantity received
unit_price	REAL	NOT NULL	Unit price at time of transaction

The stock_receipts (1) - (n) stock_receipt_items relationship correctly implements the master–detail document pattern, matching the "[For each item in items]" loop in the Stock-in Sequence Diagram.

7.3.8. Table stock_issues - Stock-out Issue (Document Header)

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Issue identifier
issue_code	TEXT	NOT NULL, UNIQUE	Stock-issue code
created_by	INTEGER	FK → users(id)	User who created the issue (audit)
issue_date	DATE	NOT NULL	Date of stock issue
recipient	TEXT	—	Recipient person/department
note	TEXT	—	Notes
created_at	DATETIME	—	Record creation timestamp

7.3.9. Table stock_issue_items — Stock-out Line Items

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Line-item identifier
issue_id	INTEGER	NOT NULL, FK → stock_issues(id)	Parent issue
product_id	INTEGER	NOT NULL, FK → products(id)	Product issued
quantity	INTEGER	NOT NULL	Quantity issued
unit_price	REAL	NOT NULL	Unit price at time of transaction

This table is the concrete implementation of the "Insert stock_issue_items (per product line)" step within the Fork/Join branch of the Stock-out Activity Diagram, executed in parallel with the products.quantity update.

7.3.10. Table activity_logs - Operation Audit Log

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Log identifier
user_id	INTEGER	-	User who performed the action (no hard FK constraint defined at the table level)
action	TEXT	-	Action content/type
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Log timestamp

Corresponds to the "commit transaction / Insert activity_logs" step in the Stock-out Activity Diagram - ensuring every successful transaction is traceable.

7.3.11. Table notifications - System Notifications

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Notification identifier
user_id	INTEGER	NOT NULL, FK → users(id)	Notification recipient

type	TEXT	NOT NULL	Notification type (e.g., stock_in, stock_out)
title	TEXT	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification content
is_read	INTEGER	NOT NULL, DEFAULT 0	Read status (0/1)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

Corresponds to the createNotification('stock_in',receiptCode)→ insertNotification() step in the Stock-in Sequence Diagram, correctly supporting the "Receive Notifications" use case shown in the Use Case Diagram.

7.3.12. Foreign Key Relationship Summary

Child Table	FK Field	Parent Table	Relationship
users	role_id	roles	N-1
products	category_id	categories	N-1
products	supplier_id	suppliers	N-1
stock_receipts	supplier_id	suppliers	N-1
stock_receipts	created_by	users	N-1
stock_receipt_items	receipt_id	stock_receipts	N-1
stock_receipt_items	product_id	products	N-1
stock_issues	created_by	users	N-1
stock_issue_items	issue_id	stock_issues	N-1
stock_issue_items	product_id	products	N-1
notifications	user_id	users	N-1

8. Implementation evidence

This section is reserved for annotated screenshots demonstrating the implemented user and administrator functions, as required by the examination's report structure. No screenshot files were supplied at the time of writing this report; the placeholders below

indicate the specific screens that should be captured, in the order recommended for a coherent demonstration.

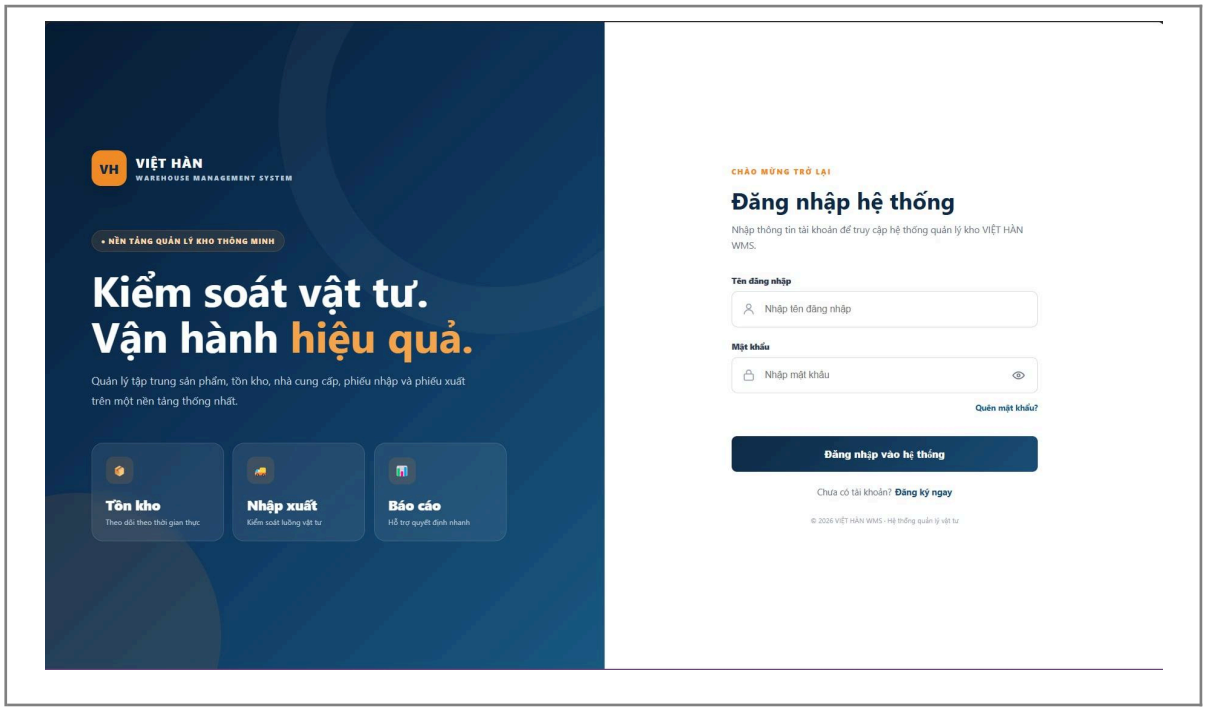

Figure 8.1. Login page

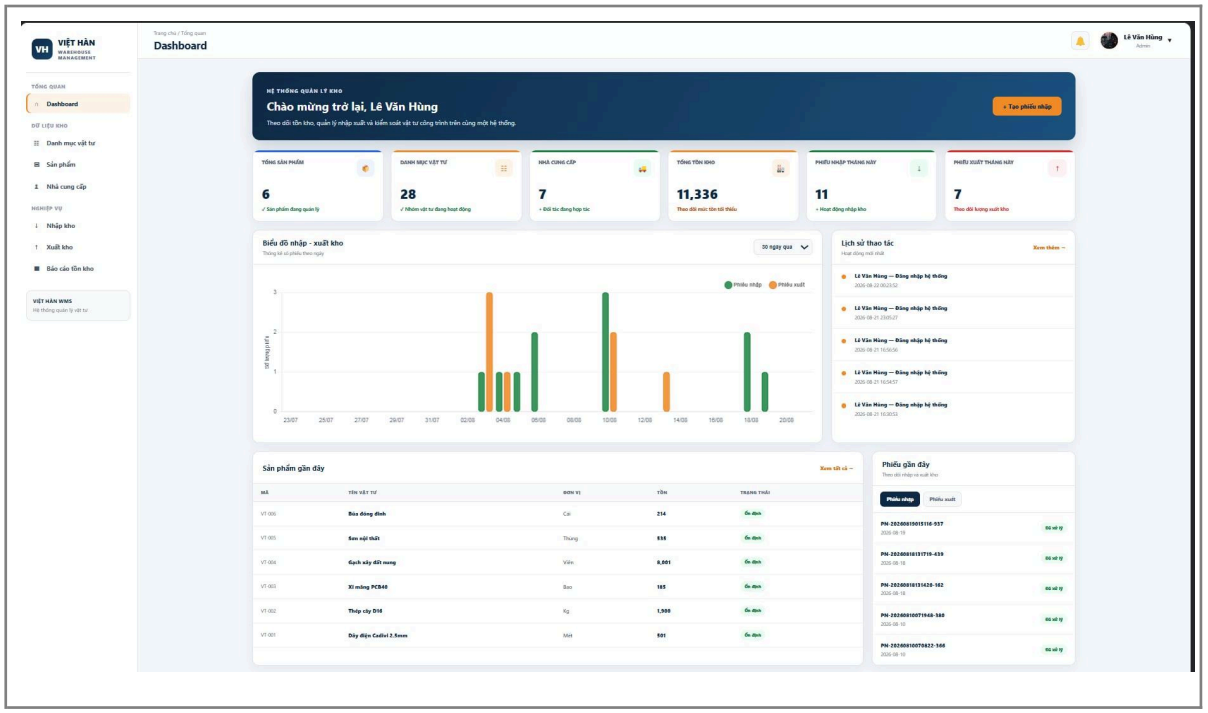

Figure 8.2. Admin dashboard

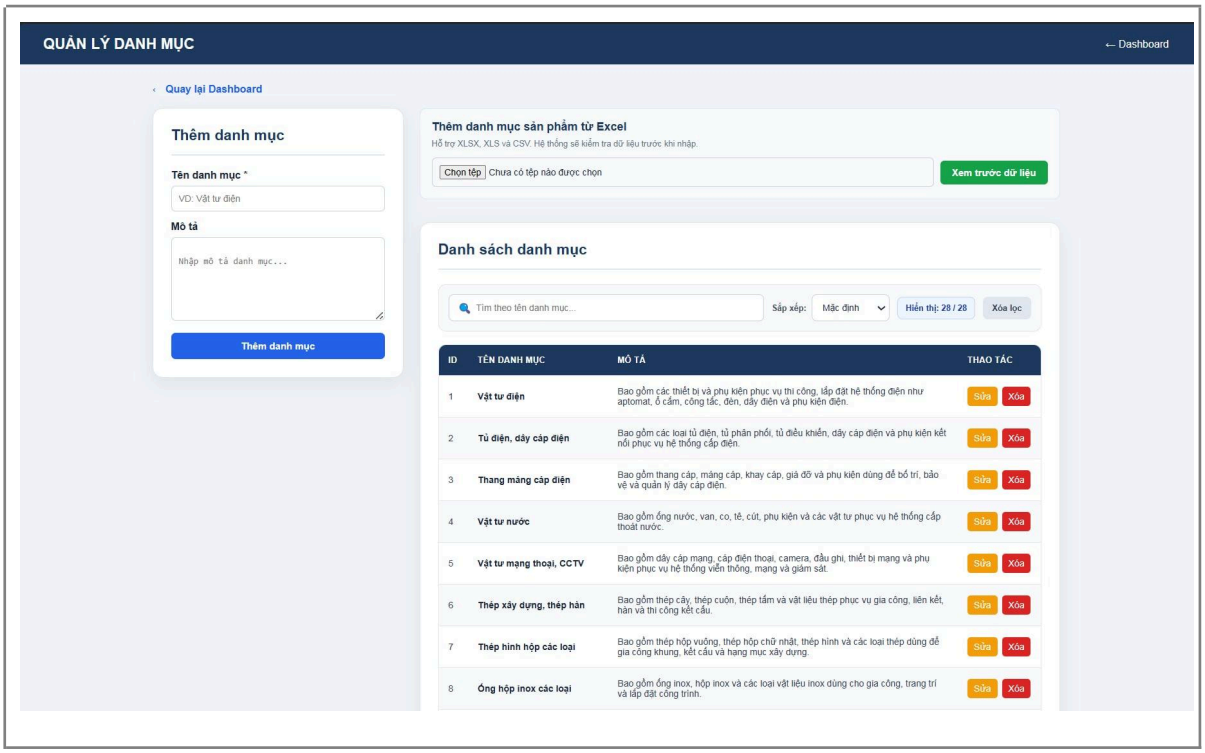

Figure 8.3. Category management (CRUD)

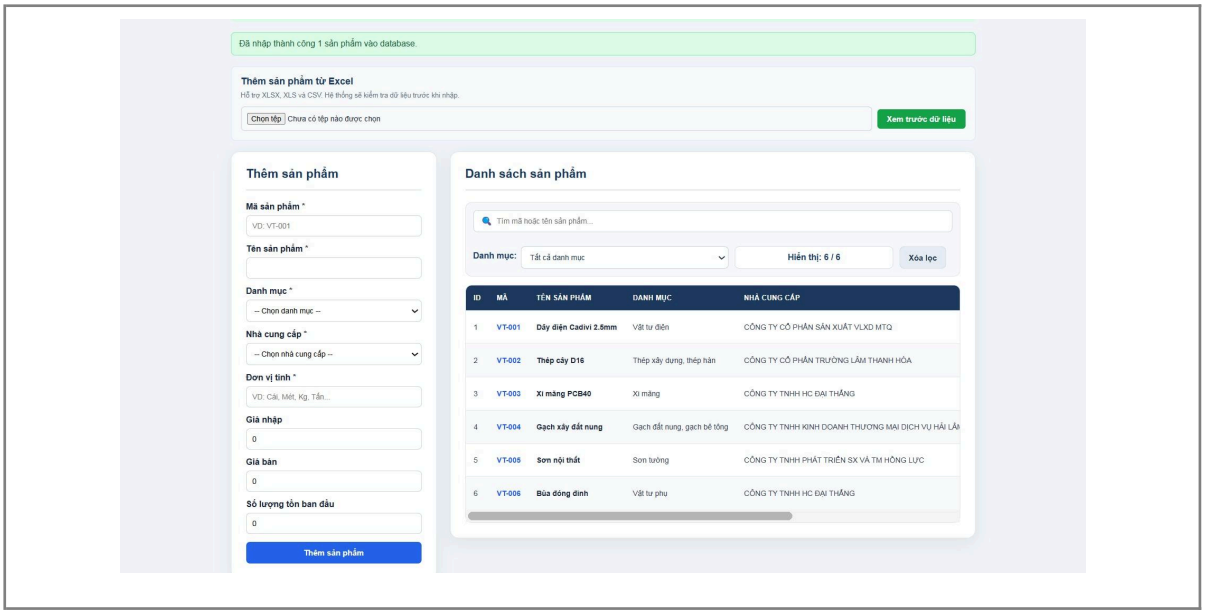

Figure 8.4. Product management (CRUD)

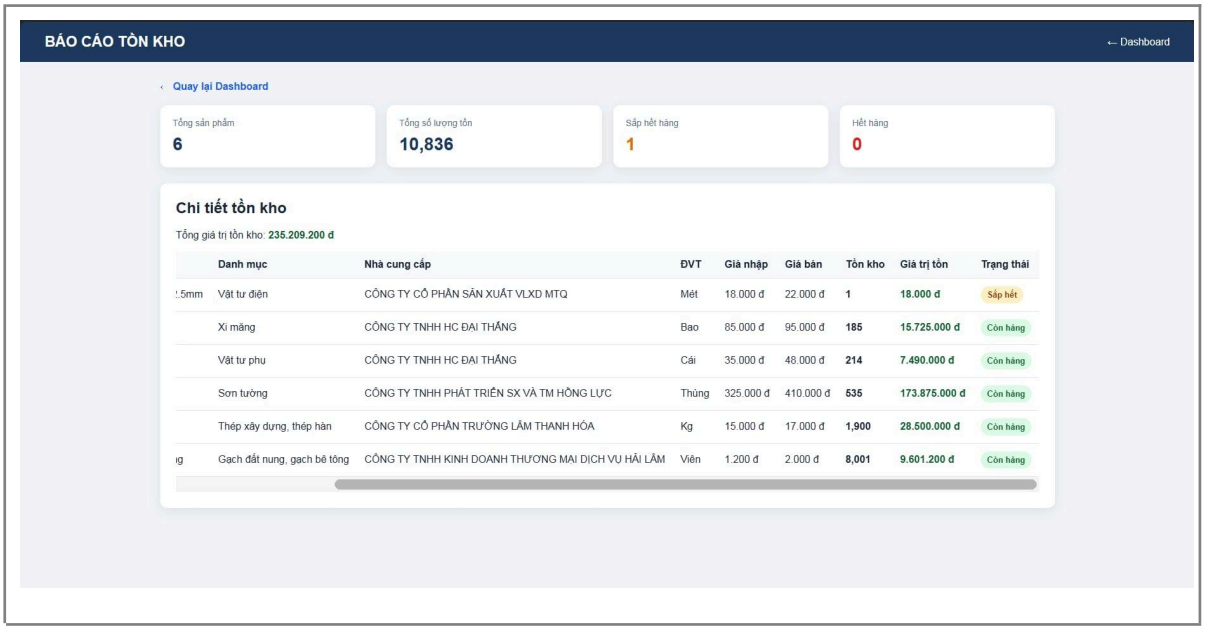

Figure 8.5. Stock-in form

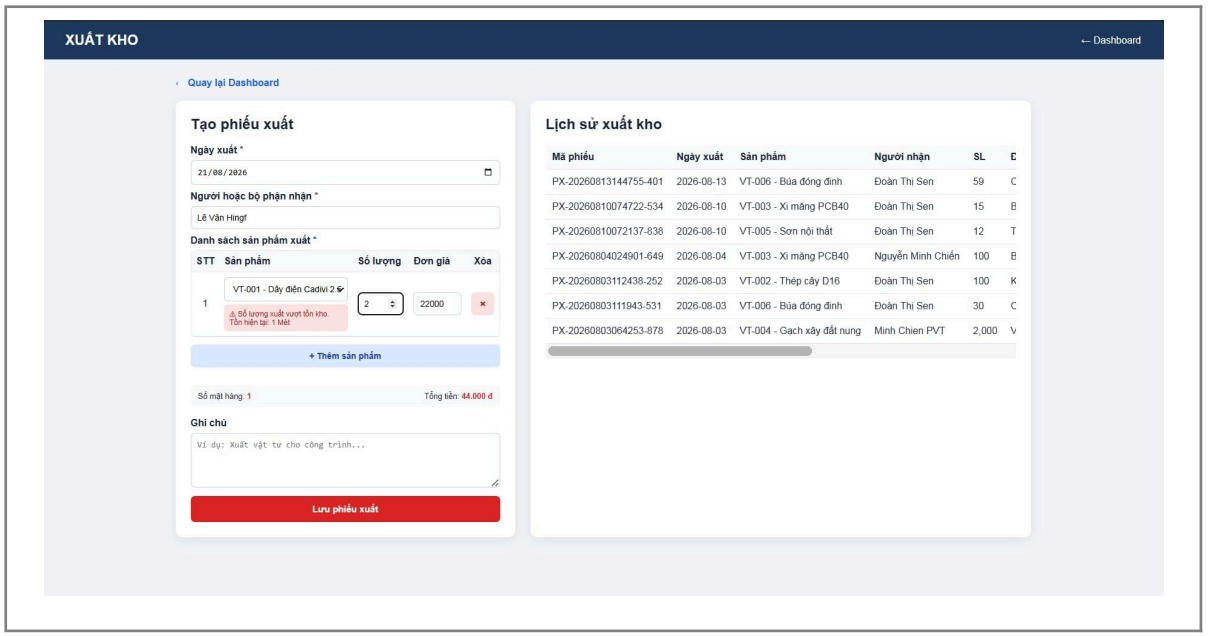

Figure 8.6. Stock-out form with insufficient-stock rejection

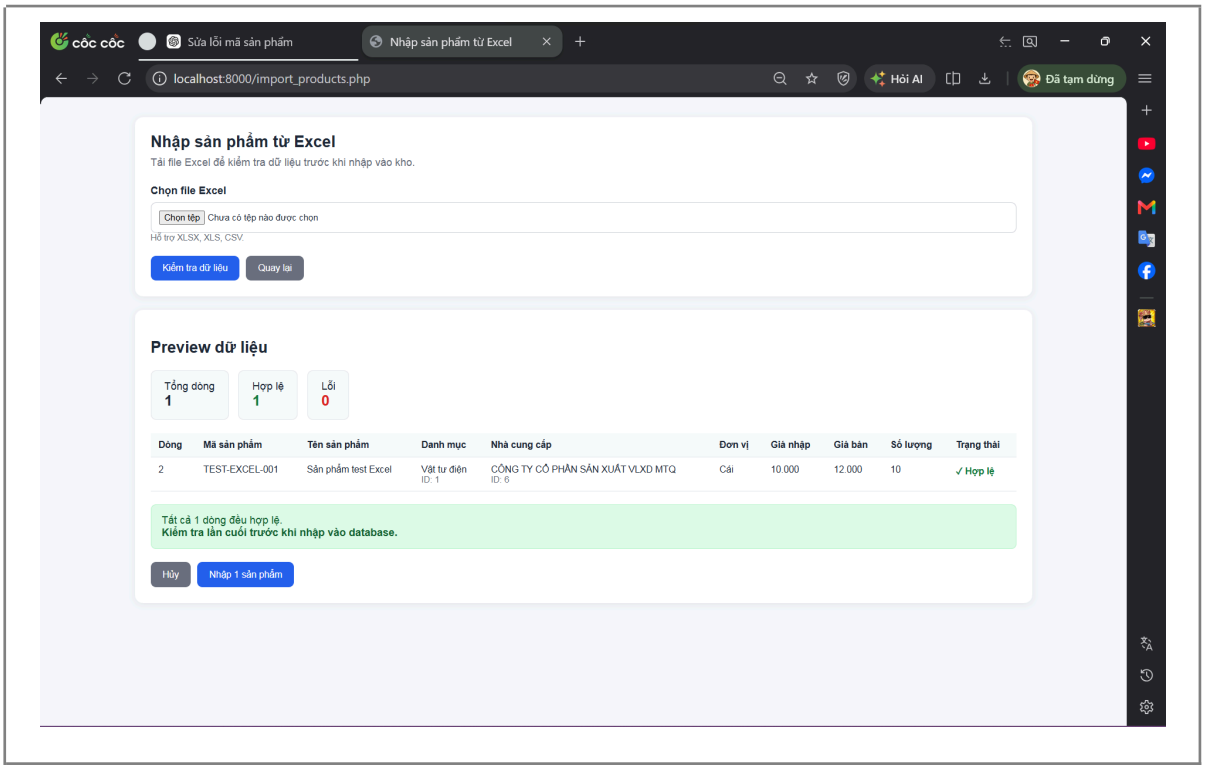

Figure 8.7. Excel import preview

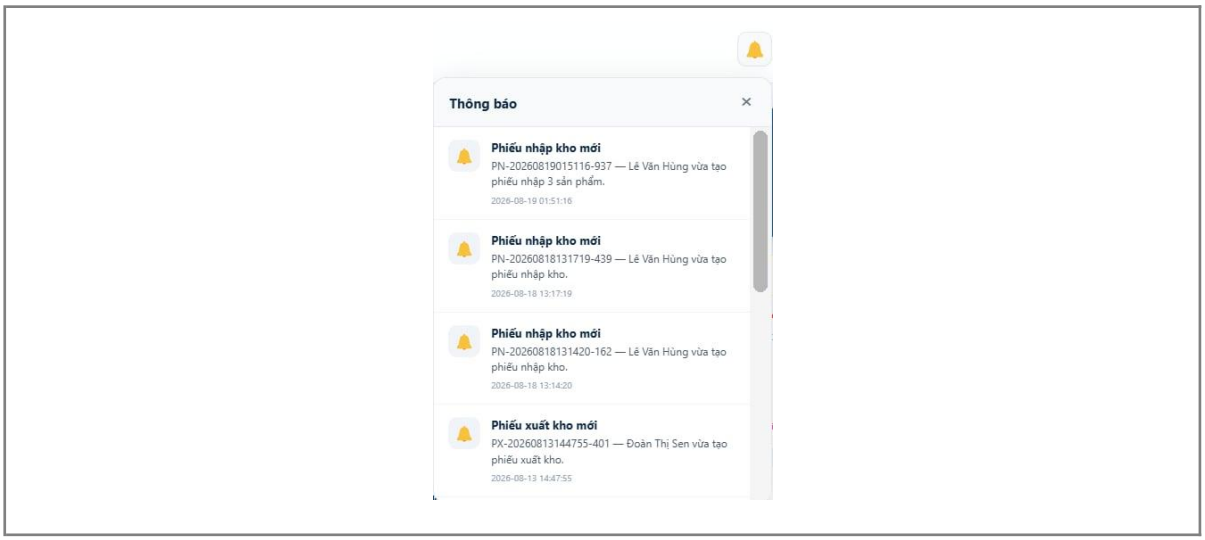

Figure 8.8. Notification panel

9. Security controls and Input-validation approach

9.1 Verified security controls

Table 9.1. Verified security controls

Control	Implementation evidence	Purpose
Password hashing	password_hash() at registration; password_verify() at login (users.password)	Prevents recovery of plaintext passwords if the database is compromised
Parameterised queries	$pdo->prepare()/execute([...]) used consistently across CRUD and transactional operations	Mitigates SQL injection by separating SQL structure from user-supplied data (OWASP Foundation, n.d.-a)
Session-based authentication and role-based authorisation	requireLogin(), isAdmin(), requireAdmin() in config/auth.php	Restricts administrative functions to the Admin role
Referential integrity	PRAGMA foreign_keys = ON in config/database.php; verified FK constraints on products, stock_receipt_items, stock_issue_items	Prevents orphaned records
Atomic, conditional stock update	UPDATE products SET quantity = quantity - ? WHERE id = ? AND quantity >= ? in stock_out.php	Prevents negative stock and mitigates race conditions between concurrent issue requests
Transactional consistency	beginTransaction()/commit()/rollBack() around multi-table writes in stock_in.php and stock_out.php	Prevents partially-written receipts/issues if an error occurs mid-operation
Output escaping	htmlspecialchars() in PHP views; escapeHtml() in notifications.js	Reduces the risk of stored/reflected cross-site scripting (XSS)

Control	Implementation evidence	Purpose
CSRF token	Present in account.php	Protects the account-update form against forged cross-origin requests (OWASP Foundation, n.d.-b)
OTP generation	random_int(100000, 999999) in forgot-password.php	Uses a cryptographically secure random source rather than rand(), which is unsuitable for security-sensitive values
Email format validation	FILTER_VALIDATE_EMAIL used in supplier import	Rejects malformed email addresses before they reach the database

9.2 Identified security weaknesses

SMTP credentials for the OTP email feature are stored in plaintext in app/config/mail.php rather than in an environment variable or an untracked configuration file. This credential should be revoked and rotated and moved out of source control before the repository is made public, in line with the examination's explicit instruction not to commit production credentials (Part III of the examination brief).

The register.php file contains hardcoded internal security codes (EMPLOYEE_SECURITY_CODE and ADMIN_SECURITY_CODE) used to control self-registration into privileged roles. If this file is exposed, an unauthorized party could potentially use the exposed codes to register an admin account.

CSRF protection is not applied uniformly throughout the application. It exists only in account.php, while other state-changing operations, including category, product, and supplier CRUD operations and some delete actions performed through HTTP GET requests, do not carry a CSRF token. This is inconsistent with current OWASP guidance that state-changing operations should require CSRF protection (OWASP Foundation, n.d.-b).

Verbose error display through error_reporting(E_ALL) and ini_set('display_errors', '1') is enabled in the import scripts. Although this configuration is appropriate for development and debugging, it should be disabled before any production or public deployment, with errors instead handled through server-side logging to avoid exposing detailed implementation information to users.

These weaknesses are presented as known limitations rather than defects that block the core functionality of the application. They should therefore be explicitly documented in the known-limitations section and addressed before any production or public deployment, in accordance with the examination's README requirements.

10. Test Plan, Test Cases, Results, and Defects

10.1. Testing Strategy

A layered testing strategy was applied to evaluate the functional correctness, business logic, data integrity, and security of the VIỆT HÀN WMS application. The testing process consisted of functional testing, boundary testing, logic testing, and security testing.

Functional testing was used to verify the main user workflows, including authentication, supplier and product management, stock-in, stock-out, and dashboard functions. Boundary testing was used to evaluate system behaviour when input values were at or beyond expected limits. Logic testing was used to verify business rules and system behaviour under different conditions, while security testing was conducted to identify potential vulnerabilities in authentication, authorization, input validation, and access control.

Integration and transaction-related testing was also considered for the stock-in and stock-out workflows, particularly to verify transaction atomicity and the prevention of invalid inventory updates. Security testing focused on the weaknesses identified during the source-code review, including privileged account registration, database exposure, CSRF protection, and input-validation weaknesses.

No performance or load testing was included in the available testing records. This is consistent with the project's intended local demonstration environment and single-warehouse scope.

10.2 Test Cases

The following test cases provide traceability between selected functional and non-functional requirements and their corresponding verification procedures.

Table 10.1. Requirement-Based Test Cases

Test ID	Requirement	Test Steps	Expected Result
TC-01	FR-02	Log in with a valid username and password	A session is created and the user is redirected to the dashboard
TC-02	FR-08, NFR-03	Attempt a stock-out with a quantity greater than the current stock	The transaction is rejected, the quantity remains unchanged, and an error message is displayed
TC-03	FR-04	Attempt to delete a category that has linked products	The deletion is blocked and an explanatory message is displayed
TC-04	FR-11	Import an Excel file containing a duplicate product code	The duplicate row is flagged during preview and excluded from the final commit
TC-05	FR-12	Enter an expired OTP during password reset	The reset request is rejected and the user is prompted to request a new OTP
TC-06	NFR-04	Force an exception during a stock-in transaction, such as using an invalid product ID	The entire transaction is rolled back and no partial receipt or quantity change remains
TC-07	FR-03	Log in as Staff and attempt to access an Admin-only URL directly, such as users.php	Access is denied and the standard access-denied response is displayed

Note: The seven test cases above represent selected requirement-based verification cases. In addition to these cases, the project testing activities included 10 primary functional test cases, 19 logic test cases, and 25 security test cases/findings.

10.3 Functional Testing Results

A total of 10 primary functional test cases were executed.

Table 10.3. Functional Testing Results

Result	Number
PASS	9
FAIL	1
Total	10

The functional testing achieved a 90% pass rate, with one failed test case requiring further investigation.

10.4 Logic Testing Results

A total of 19 logic test cases were reported as being performed.

Table 10.4. Logic Testing Results

Result	Number
PASS	6
FAIL	10
Total accounted for	16

Note: The available testing record states that 19 logic test cases were performed, but the reported PASS and FAIL results account for only 16 cases. The remaining three test cases should be verified and documented before final submission.

10.5 Security Testing Results

Security testing identified 25 security findings during the assessment.

Table 10.5. Security Testing Findings

Severity	Number of Findings
Critical	5
High	7
Medium	7

Low	6
Total	25

The findings indicate several areas requiring further security improvement, particularly input validation, privileged account registration, database protection, and consistent application of security controls.

10.6 Defects Found and Their Resolution Status

No formal defect-tracking log was maintained during the development process. However, inspection of the project repository identified evidence of iterative debugging and modification, including approximately 19 PowerShell scripts following the naming patterns patch_*.ps1 and fix_*.ps1.

These scripts provide evidence of corrective development activities involving issues such as Vietnamese character encoding and category-ID handling. Multiple .before_* versions of PHP and database files were also retained alongside the current versions, indicating that files were modified during the development and debugging process.

However, these repository artefacts should not be treated as a complete defect log, because they do not consistently document the defect ID, severity, discovery date, root cause, resolution steps, or verification results.

Table 10.6. Development Issues Identified from Repository Evidence

Evidence ID	Observed Issue	Repository Evidence	Status
DEF-E01	Vietnamese character encoding displayed incorrectly (mojibake)	patch_*.ps1 / fix_*.ps1 scripts addressing encoding-related issues	Corrective scripts identified; formal re-test result not recorded
DEF-E02	Category-ID handling required correction	patch_*.ps1 / fix_*.ps1 scripts addressing category-ID handling	Corrective scripts identified; formal re-test result not recorded

The previously retained .before_* PHP and database files are considered development artefacts rather than confirmed application defects and are therefore not included as individual defects in the table above.

Because these records were not originally maintained as a formal defect-tracking system, the severity, exact discovery date, root cause, resolution details, and test-based verification results cannot be reliably reconstructed from the repository alone. The evidence therefore demonstrates iterative debugging activity, but should not be presented as a contemporaneous defect log.

10.7 Defects Identified During Testing

In addition to the development issues identified from repository history, the testing process identified several application-level defects and security weaknesses requiring remediation. These include the following:

Table 10.7. Defects Identified During Testing

Defect ID	Observed Issue	Severity
BUG-01	Insufficient phone-number length validation; the system allows unusually long phone-number values.	Medium
BUG-02	Account persistence issue; a newly created account may not remain available during subsequent login attempts.	High
BUG-03	Integer overflow in inventory quantity; extremely large quantities may cause incorrect inventory values and potentially negative values in reports.	Critical
BUG-04	Hard-coded Admin registration code; an exposed registration code may allow unauthorized users to create an Admin account.	Critical
BUG-05	Potential direct database exposure; the SQLite database may be accessible directly if it is located within a publicly accessible web directory.	Critical

These findings originated from the testing and security assessment and are therefore distinguished from the repository-history evidence presented in Table 10.5.

10.8 Re-testing

Following the implementation of a corrective change, the available testing record confirms the following re-test result.

Table 10.8. Re-testing Results

Defect	Before Fix	After Fix
BUG-01 – Phone Number Validation	FAIL	PASS

The available evidence confirms that BUG-01 was successfully re-tested and passed after correction. The remaining logic and security findings require further remediation and re-testing before they can be considered resolved.

10.9 Testing Conclusion

The testing process demonstrated that the core functionality of VIỆT HÀN WMS is operational, with 9 out of 10 primary functional test cases passing. At the same time, logic and security testing identified several weaknesses that require further attention.

The most significant findings concern inventory quantity validation, privileged account registration, and potential database exposure. These issues should be addressed before the system is considered suitable for production deployment.

The testing results demonstrate the importance of combining functional testing with boundary, logic, and security testing, as some weaknesses may not be detected through normal functional workflows alone.

11. Installation/Setup Guide, Test Accounts, Repository Link, and Live/Local Application Link

11.1 Installation/Setup Guide

11.1.1 System Requirements

VIỆT HÀN WMS is designed to run in a local Windows environment using the PHP Built-in Development Server. The project includes a bundled PHP runtime and SQLite-related extensions, allowing the application to run without requiring a separate PHP installation.

The project uses PHP as the backend, SQLite as the relational database, and PDO for database access. The database file is located at app/database/quan_ly_kho.sqlite. MySQL is not used.

The project also uses Composer-managed dependencies, including PHPMailer 7.1.1 and PhpSpreadsheet 5.9.0.

The exact PHP version is not specified in the available project documentation.

11.1.2 Project Setup

The main application source code is located in the app/ directory. Important files include index.php, login.php, register.php, account.php, users.php, categories.php, products.php, suppliers.php, stock_in.php, stock_out.php, inventory_report.php, notifications.php, notifications_api.php, chart_data.php, forgot-password.php, and the Excel import modules. The main configuration files are located under app/config/.

The project provides a CHAY_WEB.bat file for starting the local application.

11.1.3 Database Setup

The application uses the SQLite database: *app/database/quan_ly_kho.sqlite*

The database is accessed through PDO, with foreign-key enforcement and a busy timeout configured.

The database included with the project already contains demonstration data rather than being an empty database. The documented data includes 28 categories, 7 suppliers, 7 products, 11 stock receipts, 7 stock issues, 13 receipt items, 7 issue items, 63 notifications, 98 activity logs, 7 users, and 2 roles.

11.1.4 Configuration

The main configuration files are located in app/config/, including app.php, auth.php, database.php, logger.php, mail.php, and notifications.php.

The project documentation also identifies SMTP credentials stored directly in app/config/mail.php. These credentials should not be exposed in a public repository and should be moved to an environment variable or another untracked configuration mechanism before public release.

11.1.5 Starting the Application

The application is started using the PHP Built-in Development Server. The documented command is: php\php.exe -c php\php.ini -S localhost:8000 -t app

The app directory is used as the document root.

The supplied CHAY_WEB.bat also opens reset_session.php before starting the server in order to reset the previous session for demonstration and testing purposes.

11.1.6 Accessing the Application

The documented local application address is: *http://localhost:8000/index.php*

This corresponds to app/index.php when the PHP server is started with -t app.

11.2 Test Accounts

The project uses two roles: Admin and Nhân viên (Staff). The application documentation identifies security codes that can be used during registration for these roles.

Based on the information provided by the project team, the following registration codes are available for creating test accounts:

Role	Registration Security Code	Purpose / Permissions
Admin	ADMIN2026	Administrative functions including category, product, supplier, Excel import, and account management
Nhân viên (Staff)	NV2026	Data viewing, stock-in, stock-out, and inventory reporting

These codes are registration security codes, not predefined usernames or passwords. A username and password must therefore be created during the registration process before the accounts can be used for testing.

The application documentation confirms that the system contains the Admin and Nhân viên roles and that authentication uses usernames, passwords, and session information.

11.3 Repository Link

Repository: https://github.com/met23080054-lgtm/VIET-HAN-WMS

The project documentation does not provide evidence of the repository's commit history, number of commits, branches, pull requests, or development timeline. Therefore, the repository link can be provided, but its version-control history cannot be verified from the provided Word document.

The project documentation does identify development activity through approximately 19 PowerShell scripts used for patching, fixing encoding, UI/layout, Excel import, category IDs, and other issues, but these are not equivalent to verified Git commit history.

11.4 Live/Local Application Link

Local Application: http://localhost:8000/index.php

The project is documented as a local application using the PHP Built-in Development Server on port 8000.

The local execution procedure is:

1. Open the VIỆT HÀN WMS project directory.
2. Use the bundled PHP runtime included in the project.
3. Run CHAY_WEB.bat, or execute the documented PHP server command.
4. Allow the project to start the PHP Built-in Development Server on port 8000.
5. Open http://localhost:8000/index.php in a web browser.
6. Create or use a test account with the appropriate role.
7. Test the corresponding functions for Admin or Nhân viên.

The project also includes the PHP runtime and SQLite extensions required for the local environment, which supports the documented local execution approach.

Information Verification

Information	Exact Information	Status
PHP version	Not specified in the Word document	Missing
Backend	PHP	Verified
Database	SQLite	Verified
Database name	quan_ly_kho.sqlite	Verified
Database path	app/database/quan_ly_kho.sqlite	Verified
SQL file	Not documented	Missing
Server	PHP Built-in Development Server	Verified
Server command	php\php.exe -c php\php.ini -S localhost:8000 -t app	Verified
Local URL	http://localhost:8000/index.php	Verified
Admin registration code	ADMIN2026	Provided by project team
Staff registration code	NV2026	Provided by project team

Admin username/password	Created during registration	Not predetermined
Staff username/password	Created during registration	Not predetermined
Repository URL	https://github.com/met23080054-lgtm/VIET-HAN-WMS	Provided

12. Requirement traceability and Objective achievement

12.1 Traceability matrix

Table 12.1. Requirement traceability matrix

Requirement	Design	Implementation	Test case
FR-02 (Login)	Use case: Log in	login.php	TC-01
FR-08 (Stock-out, no negative stock)	Activity diagram, Section 6.2	stock_out.php	TC-02
FR-04 (Category delete protection)	Use case: Manage categories	categories.php	TC-03
FR-11 (Excel import)	Use case: Import Excel data	import_products.php	TC-04
FR-12 (OTP reset)	Use case: Reset password via OTP	forgot-password.php	TC-05
NFR-04 (Transaction atomicity)	Sequence diagram, Section 6.3	stock_in.php, stock_out.php	TC-06
FR-03 (Role restriction)	Use case diagram, Section 6.1	config/auth.php	TC-07

12.2 Achievement of objectives

Table 12.2. Achievement of objectives

Objective	Evidence	Achievement
Digitalize master data management (categories, products, suppliers)	Verified CRUD in categories.php, products.php, suppliers.php with delete-protection rules	Achieved
Provide a reliable stock-in/stock-out workflow with data integrity	Verified transaction + conditional update logic in stock_in.php/stock_out.php	Achieved
Provide role-based access control	Verified requireAdmin()/isAdmin() and roles table	Achieved
Provide reporting and dashboard visibility	Verified index.php, inventory_report.php, chart_data.php	Achieved
Provide bulk data import	Verified import_*.php with preview-before-commit design	Achieved
Provide adequate security for a production deployment	Password hashing and prepared statements verified; CSRF and hardcoded-credential gaps identified	Partially achieved - remediation required before production use
Provide automated testing evidence	No automated test suite or executed test log found	Not achieved - see Section 10

13. Results and Discussion

Measured against the manual or spreadsheet-based inventory processes that the system is intended to replace, VIỆT HÀN WMS demonstrably centralises stock records in a relational database rather than in disconnected spreadsheet files, and it enforces two integrity rules - the atomic, quantity-checked stock-out update and the transactional wrapping of multi-table writes - that a spreadsheet-based process cannot guarantee. This is consistent with the broader finding in the WMS literature that structured, system-enforced

controls reduce the inventory discrepancies associated with manual data entry (Atieh et al., 2016).

The most significant technical strength verified in this report is not the visual interface but the completeness of the underlying business workflow: authentication and authorization, master-data CRUD with referential-integrity protection, a header-detail transaction model for both receipts and issues, atomic stock control, Excel import with a validate-then-preview-then-commit sequence, and supporting features such as notifications and activity logging. Conversely, the most significant gaps relative to the HSB2006 marking scheme are process-related rather than functional: the absence of a documented, pre-implementation requirements artifact, UML diagrams rendered as formal figures, executed test evidence, and a GitHub history showing incremental development. These are addressable without further coding, since the underlying functionality already exists and has been verified.

14. Conclusion and Recommendations

VIỆT HÀN WMS satisfies the core technical expectations of the HSB2006 examination for the Database and Backend Development criterion: a working relational database, PHP-to-database connectivity via PDO, authentication, role-based authorization, CRUD operations, and at least one complete, integrity-protected business workflow (stock-in and stock-out). The remaining work before submission is concentrated in documentation and process evidence rather than in the application itself.

15. References

Atieh, A. M., Kaylani, H., Al-abdallat, Y., Qaderi, A., Ghoul, L., Jaradat, L., & Hamdan, I. (2016). Performance improvement of inventory management system processes by an automated warehouse management system. *Procedia CIRP, 41*, 568-572. https://doi.org/10.1016/j.procir.2015.12.122

Hanoi School of Business and Management, Vietnam National University, Hanoi. (2025). *HSB2006 - Developing Business Applications: Final examination brief (Semester 2, AY 2024-2025)*.

OWASP Foundation. (n.d.-a). *Cross-site request forgery prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

16. AI/Tool-Use declaration

The examination brief (Part VI) requires an explicit declaration of any AI or coding-assistant tools used during development, to be completed truthfully by the student(s). The template below must be filled in accurately; it has intentionally not been pre-filled with invented content.

Table 16.1. AI/Tool-use declaration

Field	Response
AI/tool name(s)	ChatGPT, GitHub Copilot, Claude
Purpose of use	Supporting PHP debugging, suggesting SQL query structures, and writing form validation
Main parts assisted	Module stock_in.php, forgot-password.php (OTP email), import Excel
How the output was verified	Manually test on the local server, check the results in SQLite Browser, and cross-check with business requirements

This report itself was produced with the assistance of an AI writing tool, based on direct inspection of the submitted source code and database file; this fact should also be disclosed by the student, together with a description of how each generated claim in this report was checked against the actual code (a process the student can observe was applied throughout, e.g., Sections 6.4 and 9, where every figure was cross-checked against PRAGMA queries or literal source-code excerpts before being written).

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
ERP	Enterprise Resource Planning

OTP	One-Time Password
PDO	PHP Data Objects
RFID	Radio-Frequency Identification
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
WMS	Warehouse Management System
XSS	Cross-Site Scripting